```
### codility/cheatsheet.py

"""
CODILITY CHEATSHEET: read before the test

1. Before you code
    - Read the ASSUMPTIONS. N up to 100,000 -> need O(N) or O(N log N).
      O(N^2) passes correctness but fails performance (~50% score).
    - Note the caps: "return 1,000,000,000 if larger", "return -1 if ...".
    - Split the time: count the tasks at the start, give each an equal share.
    - Correct brute force first (partial score), then optimise.

2. Edge cases Codility always tests
    - empty / single element / two elements
    - all negative, all equal, all zero
    - largest N, largest values, values at the range limits
    - answer at index 0 or at the last index

3. Pick the technique from the question
    | Question says                                | Try                          |
    |----------------------------------------------|------------------------------|
    | sum / count over a range, many queries       | prefix sums                  |
    | max sum of a slice, best buy/sell            | Kadane (max slice)           |
    | value that appears in more than half         | leader (Boyer-Moore)         |
    | slices with a property, pairs in sorted arr  | two pointers / caterpillar   |
    | "minimal largest ...", "smallest k such"     | binary search on the answer  |
    | matching pairs, nesting, "survivors"         | stack                        |
    | best of a few products / triangles           | sort first                   |
    | reach the end with jumps / dice / steps      | 1-D DP                       |
    | missing / seen / count of values             | set or count array           |
    | top k, repeatedly take the smallest          | heapq                        |
    | divisors, primes                             | loop to sqrt(n), sieve       |

4. My own common bugs (from this week)
    1. Bounds: valid indices are 0..n-1 -> check `== n`, and check bounds BEFORE indexing.
    2. `return` inside a loop cuts off the other branches.
    3. Wrong variable: start cell vs popped cell, (r,c) vs the neighbour.
    4. Inverted condition: name what you KEEP (`!= 1`), don't list what to skip.
    5. Count once per LEVEL vs once per item; off-by-one in totals (start at 0 or 1?).
    6. Changing a pointer before saving what you need (save the node first).
    7. Comparing ints with strings ("0" vs 0): check the input type.
"""

import heapq
from math import isqrt


# ---------------------------------------------------------------------------
# PREFIX SUMS: O(N) build, O(1) per range query  (PassingCars, GenomicRangeQuery)
# ---------------------------------------------------------------------------
def prefix_sums(A):
    P = [0] * (len(A) + 1)              # P[i] = sum of A[0:i]  (size N+1, P[0] = 0)
    for i, x in enumerate(A):
        P[i + 1] = P[i] + x
    return P


def range_sum(P, left, right):          # sum of A[left..right], both inclusive
    return P[right + 1] - P[left]       # (!) right+1: P is exclusive


# ---------------------------------------------------------------------------
# KADANE / MAX SLICE: O(N)  (MaxSliceSum, MaxProfit)
# ---------------------------------------------------------------------------
def max_slice_sum(A):                   # non-empty slice; works for all-negative A
    best = cur = A[0]
    for x in A[1:]:
        cur = max(x, cur + x)           # extend the slice, or start fresh at x
        best = max(best, cur)
    return best


def max_profit(prices):                 # buy once, sell later; 0 if no profit
    lowest, best = float("inf"), 0
    for p in prices:
        lowest = min(lowest, p)         # cheapest buy so far
        best = max(best, p - lowest)    # sell today
```

```
    return best


def max_double_slice_sum(A):              # X < Y < Z, sum of A[X+1..Y-1] + A[Y+1..Z-1]
    n = len(A)
    left, right = [0] * n, [0] * n        # best slice ending at i / starting at i (may be empty)
    for i in range(1, n - 1):
        left[i] = max(0, left[i - 1] + A[i])
    for i in range(n - 2, 0, -1):
        right[i] = max(0, right[i + 1] + A[i])
    return max(left[y - 1] + right[y + 1] for y in range(1, n - 1))


# ---------------------------------------------------------------------------
# LEADER (Boyer-Moore voting): O(N) time, O(1) space  (Dominator, EquiLeader)
# ---------------------------------------------------------------------------
def leader(A):                            # value in MORE than half of A, else None
    candidate, count = None, 0
    for x in A:
        if count == 0:
            candidate, count = x, 1
        elif x == candidate:
            count += 1
        else:
            count -= 1                    # a pair of different values cancels out
    # (!) the candidate is only a guess: always verify it
    return candidate if A and A.count(candidate) > len(A) // 2 else None


def equi_leaders(A):                      # splits where both sides share the same leader
    lead = leader(A)
    if lead is None:
        return 0
    total, left, res = A.count(lead), 0, 0
    for i in range(len(A) - 1):
        left += A[i] == lead
        right = total - left
        if left > (i + 1) // 2 and right > (len(A) - i - 1) // 2:
            res += 1
    return res


# ---------------------------------------------------------------------------
# TWO POINTERS / CATERPILLAR: O(N)  (CountDistinctSlices, AbsDistinct)
# ---------------------------------------------------------------------------
def count_distinct_slices(A, cap=1_000_000_000):
    seen, left, res = set(), 0, 0
    for right, x in enumerate(A):
        while x in seen:                  # shrink from the left until x is unique
            seen.remove(A[left])
            left += 1
        seen.add(x)
        res += right - left + 1           # every slice ending at `right`
        if res > cap:
            return cap
    return res


def count_triangles(A):                   # triplets (sorted) with a + b > c: O(N^2)
    A = sorted(A)
    res = 0
    for x in range(len(A)):
        z = x + 2
        for y in range(x + 1, len(A)):
            while z < len(A) and A[x] + A[y] > A[z]:
                z += 1                    # z never moves back: caterpillar
            res += z - y - 1
    return res


# ---------------------------------------------------------------------------
# BINARY SEARCH ON THE ANSWER: O(N log(range))  (MinMaxDivision, Koko)
# ---------------------------------------------------------------------------
def min_max_division(K, A):               # split A into <= K blocks, minimise the largest sum
    def blocks_needed(limit):
        blocks, cur = 1, 0
        for x in A:
```

```
            if cur + x > limit:
                blocks, cur = blocks + 1, x
            else:
                cur += x
        return blocks

    lo, hi = max(A), sum(A)              # answer is somewhere in [max, sum]
    while lo < hi:
        mid = (lo + hi) // 2
        if blocks_needed(mid) <= K:
            hi = mid                     # mid works: try smaller
        else:
            lo = mid + 1                 # mid too small
    return lo


# ---------------------------------------------------------------------------
# STACKS  (Brackets, Fish, StoneWall)
# ---------------------------------------------------------------------------
def brackets(S):
    pairs, stack = {")": "(", "]": "[", "}": "{"}, []
    for ch in S:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return 0
        else:
            stack.append(ch)
    return 1 if not stack else 0         # (!) leftover openers -> not nested


def fish(A, B):                          # B[i] = 1 swims downstream, 0 upstream
    down, alive = [], 0
    for size, d in zip(A, B):
        if d == 1:
            down.append(size)
        else:
            while down and down[-1] < size:
                down.pop()               # upstream fish eats smaller downstream ones
            if not down:
                alive += 1               # survived all of them
    return alive + len(down)


def stone_wall(H):
    stack, blocks = [], 0
    for h in H:
        while stack and stack[-1] > h:
            stack.pop()                  # taller blocks end here
        if not stack or stack[-1] < h:
            stack.append(h)              # need a new block at this height
            blocks += 1
    return blocks


# ---------------------------------------------------------------------------
# SORTING TRICKS  (Distinct, MaxProductOfThree, Triangle)
# ---------------------------------------------------------------------------
def max_product_of_three(A):
    A = sorted(A)
    # (!) two big negatives times the largest positive can win
    return max(A[-1] * A[-2] * A[-3], A[0] * A[1] * A[-1])


def triangle(A):                         # any triplet with sides forming a triangle?
    A = sorted(A)
    return int(any(A[i] + A[i + 1] > A[i + 2] for i in range(len(A) - 2)))


# ---------------------------------------------------------------------------
# 1-D DP  (NumberSolitaire, House Robber, Climbing Stairs)
# ---------------------------------------------------------------------------
def number_solitaire(A):                 # from square 0 to N-1, throw a die (1..6)
    dp = [A[0]] + [float("-inf")] * (len(A) - 1)
    for i in range(1, len(A)):
        dp[i] = A[i] + max(dp[max(0, i - 6):i])    # best of the previous 6 squares
    return dp[-1]
```

```python
def house_robber(nums):                 # no two adjacent houses
    rob, skip = 0, 0                    # best total if we rob / skip the previous house
    for x in nums:
        rob, skip = skip + x, max(rob, skip)
    return max(rob, skip)


# ---------------------------------------------------------------------------
# COUNTING / SETS  (MissingInteger, PermCheck, MaxCounters)
# ---------------------------------------------------------------------------
def missing_integer(A):                 # smallest positive not in A: answer is in 1..N+1
    seen = set(A)
    for i in range(1, len(A) + 2):
        if i not in seen:
            return i


def perm_check(A):                      # is A a permutation of 1..N?
    return int(set(A) == set(range(1, len(A) + 1)))


# ---------------------------------------------------------------------------
# MATH, BITS, HEAP
# ---------------------------------------------------------------------------
def count_factors(n):                   # O(sqrt n)
    res = 0
    for i in range(1, isqrt(n) + 1):
        if n % i == 0:
            res += 1 if i * i == n else 2    # i and n // i
    return res


def sieve(n):                           # primes up to n: O(n log log n)
    is_prime = [False, False] + [True] * (n - 1)
    for i in range(2, isqrt(n) + 1):
        if is_prime[i]:
            for j in range(i * i, n + 1, i):
                is_prime[j] = False
    return [i for i, p in enumerate(is_prime) if p]


def count_div(A, B, K):                 # multiples of K in [A, B], O(1)
    return B // K - (A - 1) // K if A > 0 else B // K + 1


# Bits:   n & 1 (odd?)   n >> 1 (halve)   n & (n - 1) (drop lowest 1)   bin(n).count("1")
# Heap:   heapq.heappush(h, x)   heapq.heappop(h)   push -x for a MAX heap
#         heapq.heapify(list) O(n)   heapq.nlargest(k, A)   heapq.nsmallest(k, A)
# GCD:    from math import gcd     lcm = a * b // gcd(a, b)


if __name__ == "__main__":              # every template above, on Codility's examples
    assert range_sum(prefix_sums([1, 2, 3, 4]), 1, 2) == 5
    assert max_slice_sum([3, 2, -6, 4, 0]) == 5 and max_slice_sum([-10, -3, -5]) == -3
    assert max_profit([23171, 21011, 21123, 21366, 21013, 21367]) == 356
    assert max_double_slice_sum([3, 2, 6, -1, 4, 5, -1, 2]) == 17
    assert leader([3, 4, 3, 2, 3, -1, 3, 3]) == 3 and leader([1, 2]) is None
    assert equi_leaders([4, 3, 4, 4, 4, 2]) == 2
    assert count_distinct_slices([3, 4, 5, 5, 2]) == 9
    assert count_triangles([10, 2, 5, 1, 8, 12]) == 4
    assert min_max_division(3, [2, 1, 5, 1, 2, 2, 2]) == 6
    assert brackets("{[()()]}") == 1 and brackets("([)()]") == 0 and brackets("") == 1
    assert fish([4, 3, 2, 1, 5], [0, 1, 0, 0, 0]) == 2
    assert stone_wall([8, 8, 5, 7, 9, 8, 7, 4, 8]) == 7
    assert max_product_of_three([-3, 1, 2, -2, 5, 6]) == 60
    assert max_product_of_three([-5, -6, 1, 2, 3]) == 90
    assert triangle([10, 2, 5, 1, 8, 20]) == 1 and triangle([10, 50, 5, 1]) == 0
    assert number_solitaire([1, -2, 0, 9, -1, -2]) == 8
    assert house_robber([2, 7, 9, 3, 1]) == 12
    assert missing_integer([1, 3, 6, 4, 1, 2]) == 5 and missing_integer([-1, -3]) == 1
    assert perm_check([4, 1, 3, 2]) == 1 and perm_check([4, 1, 3]) == 0
    assert count_factors(24) == 8 and count_factors(36) == 9
    assert sieve(20) == [2, 3, 5, 7, 11, 13, 17, 19]
    assert count_div(6, 11, 2) == 3 and count_div(0, 0, 11) == 1
    print("all cheatsheet templates pass")
```